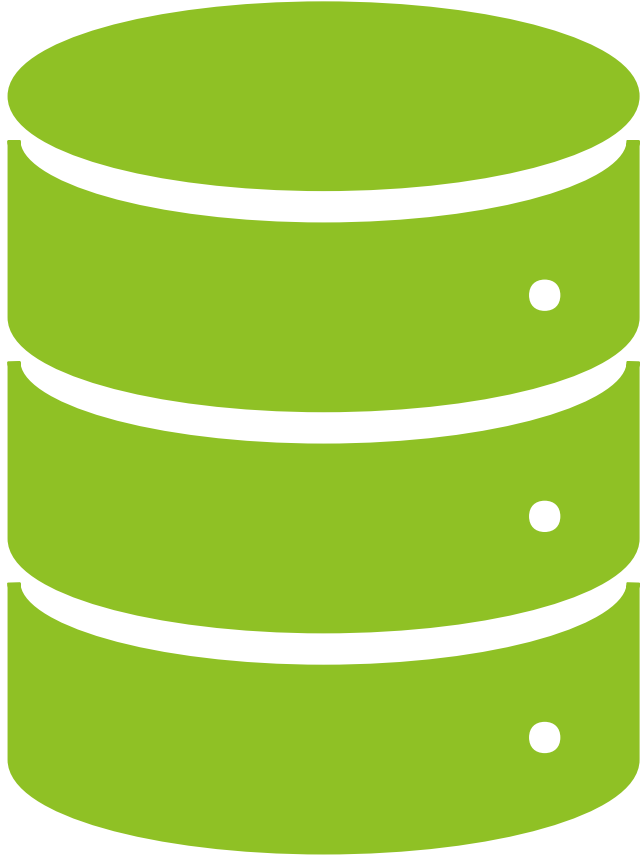




Python + SQL Project

By: Deepti Singh



OBJECTIVE

- This project analyses an e-commerce dataset, using SQL for data extraction and querying and Python for advanced analysis and visualization. The objective is to generate actionable business insights across sales performance, customer behaviour, and customer retention, supporting data-driven decision-making.



For the **Python-based data analysis**, I cleaned the dataset by merging all tables into a single consolidated table. I also removed duplicate records and eliminated unnecessary data to ensure accuracy and consistency before performing the analysis.



For the **SQL-based data analysis**, I uploaded the tables in their original structure without modification and conducted the analysis directly using SQL queries.

Methodology for Python and SQL Data Analysis



BASIC PROBLEMS



A. List all unique cities where customers are located

Python Code

```
Unique_customer_city = sorted(df_customers['customer_city'].unique())
print("Number of Unique Cities where Customers are located:", len(Unique_customer_city))

for city in Unique_customer_city:
    print(city)
```

```
Number of Unique Cities where Customers are located: 4119
abadia dos dourados
abadiania
abaete
abaetetuba
abaiara
abaira
abare
abatia
abdon batista
abelardo luz
abranter
abre campo
abreu e lima
acaiaca
acailandia
acajutiba
acarau
```

SQL Code

```
/*1. List all unique cities where customers are located. */
select distinct(customer_city) from customers
order by customer_city;
```

	customer_city
▶	Abadia Dos Dourados
	Abadiania
	Abaete
	Abaetetuba
	Abaiara
	Abaira
	Abare
	Abatia
	Abdon Batista
	Abelardo Luz
	Abrantes

Result 1 x

Output

B. Count the number of orders placed in 2017

Python Code

```
orders_2017= (df_merged[df_merged['order_purchase_timestamp'].dt.year== 2017]).copy()  
orders_2017  
print("Count of Orders placed in 2017: ", len(orders_2017))  
unique_orders_count = orders_2017['order_id'].nunique()  
print("Count of Unique Orders placed in 2017: ", unique_orders_count)
```

Count of Orders placed in 2017: 54105

Count of Unique Orders placed in 2017: 45101



SQL Code

```
select year(order_purchase_timestamp), count(order_id) from orders  
where year(order_purchase_timestamp) = 2017  
group by year(order_purchase_timestamp);
```

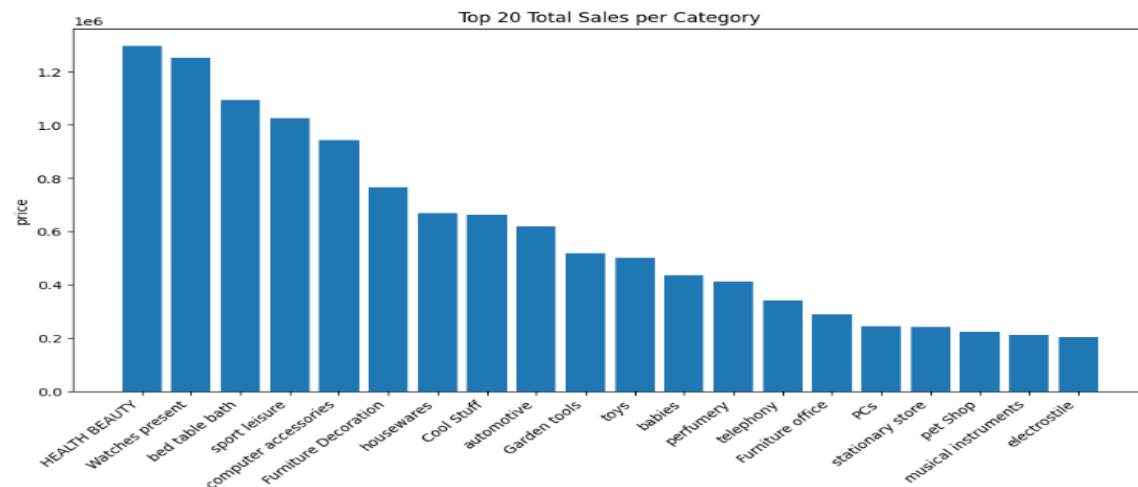
	year(order_purchase_timestamp)	count(order_id)
►	2017	45101

C. Find the total sales per category

Python Code

```
sales_per_category= df_merged.groupby('product_category')['price'].sum().reset_index()
sales_per_category= sales_per_category.sort_values(by='price', ascending=False)
sales_per_category
```

	product_category	total_sales
30	HEALTH BEAUTY	1258681.34
45	Watches present	1205005.68
49	bed table bath	1036988.68
68	sport leisure	988048.97
53	computer accessories	911954.32



SQL Code

```
select p.product_category, round(sum(o.price), 2) as total_sales from order_items as o
join products as p
on o.product_id = p.product_id
group by p.product_category
order by total_sales desc;
```

product_category	total_sales
Health Beauty	1258681.34
Watches Present	1205005.68
Bed Table Bath	1036988.68
Sport Leisure	988048.97
Computer Accessories	911954.32
Furniture Decoration	729762.49
Cool Stuff	635290.85
Housewares	632248.66
Automotive	592720.11
Garden Tools	485256.46
Toys	483946.6

D. Calculate the percentage of orders that were paid in instalments

Python Code

```
#Total Number of Unique Orders
total_no_of_orders= df_merged['order_id'].nunique()
total_no_of_orders

#Total Number of Unique Orders paidin Installments
orders_installments= df_merged[df_merged['payment_installments'].notna()
& (df_merged['payment_installments'] > 1)]['order_id'].nunique()
orders_installments

#percentage of Orders that were paid in installments
percentage_installments= (orders_installments/total_no_of_orders)*100
percentage_installments= round(percentage_installments, 2)
percentage_installments

print("Total Number of Unique Orders Placed: ", total_no_of_orders)
print("Total Number of Unique Orders Paid in Installments: ", orders_installments)
print("Percentage of Orders Paid in Installments: ", percentage_installments,"%")
```

```
Total Number of Unique Orders Placed: 99441
Total Number of Unique Orders Paid in Installments: 51170
Percentage of Orders Paid in Installments: 51.46 %
```

SQL Code

```
SELECT
    ROUND(COUNT(DISTINCT CASE
        WHEN payment_installments > 1 THEN order_id
        END) * 100 / COUNT(DISTINCT order_id),
    2) AS Percentage_installments_orders
FROM
    payments;
```

	Percentage_installments_orders
▶	51.46

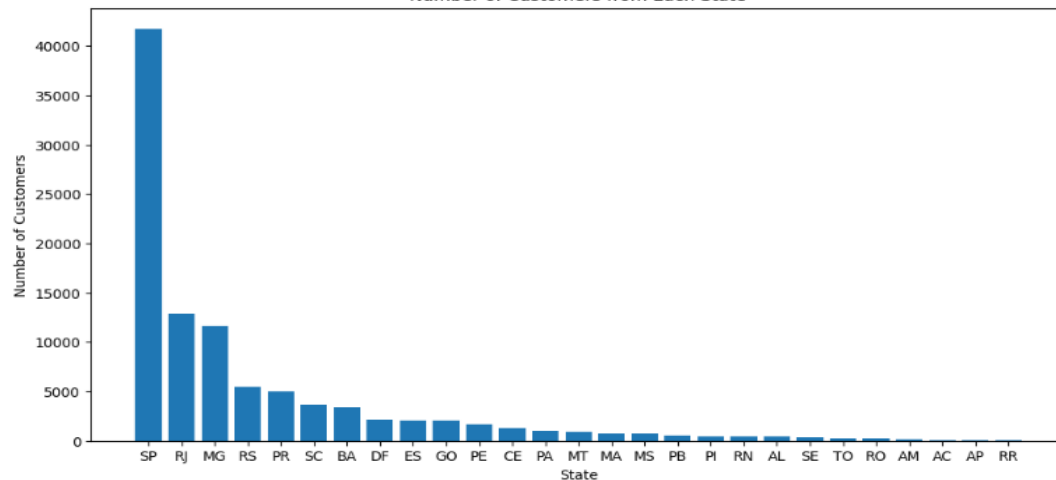
E. Count the number of customers from each state

Python Code

```
customer_state_counts= df_customers['customer_state'].value_counts().reset_index()  
customer_state_counts
```

	customer_state	count
0	SP	41746
1	RJ	12852
2	MG	11635
3	RS	5466
4	PR	5045
5	SC	3637
6	BA	3380
7	DF	2140

Number of Customers from Each State



SQL Code

```
select customer_state, count(customer_id) as total_customers  
from customers  
group by customer_state  
order by count(customer_id) desc;
```

customer_state	total_customers
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140
ES	2033



INTERMEDIATE PROBLEMS



A. Calculate the number of orders per month in 2018

Python Code

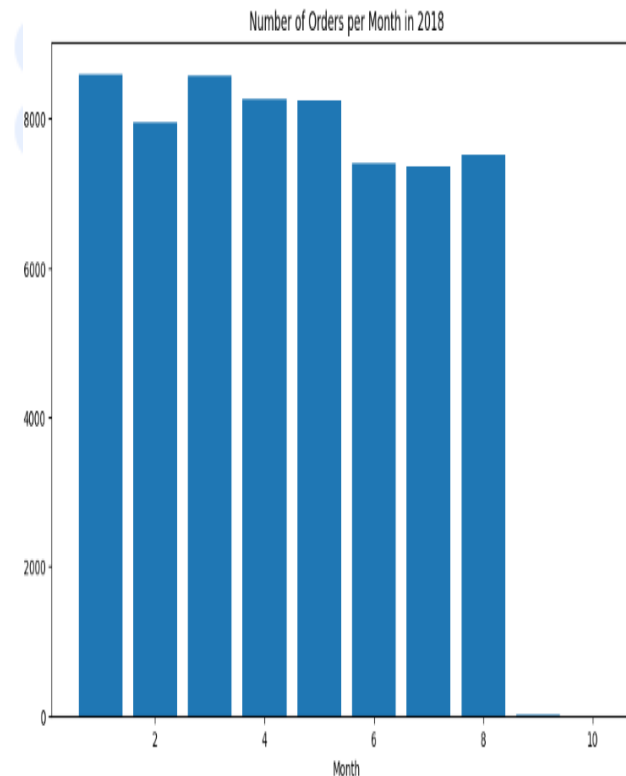
```
orders_2018= df_merged[df_merged['order_purchase_timestamp'].dt.year== 2018]

order_count_2018= orders_2018.groupby(orders_2018['order_purchase_timestamp'].
dt.month).size().reset_index(name='order_count')

order_count_2018
```

order_purchase_timestamp	order_count
--------------------------	-------------

0	1	7269
1	2	6728
2	3	7211
3	4	6939
4	5	6873
5	6	6167
6	7	6292
7	8	6512
8	9	16
9	10	4



SQL Code

```
select extract(year from order_purchase_timestamp) as Year_extracted,
       extract(month from order_purchase_timestamp) as month_extracted,
       count(customer_id) as Number_of_customers
from orders
where extract(year from order_purchase_timestamp) = 2018
group by Year_extracted, month_extracted
order by month_extracted;
```

Year_extracted	month_extracted	Number_of_customers
2018	1	7269
2018	2	6728
2018	3	7211
2018	4	6939
2018	5	6873
2018	6	6167
2018	7	6292
2018	8	6512
2018	9	16
2018	10	4

B. Find the average number of products per order, grouped by customer city.

Python Code

```
avg_products_per_city= df_merged.groupby(['customer_city', 'order_id'])['product_id'].count().reset_index(name='avg_products_per_order_per_city' )

avg_products_per_city = avg_products_per_city.groupby('customer_city')['avg_products_per_order_per_city'].mean().reset_index()

avg_products_per_city = avg_products_per_city.sort_values(by='avg_products_per_order_per_city',
                                                         ascending=False)

print("Average number of products per order, grouped by customer city:")
avg_products_per_city
```

Average number of products per order, grouped by customer city:

	customer_city	avg_products_per_order_per_city
2625	padre carvalho	7.0
909	celso ramos	6.5
758	candido godoi	6.0
1156	datas	6.0
2269	matias olimpio	5.0

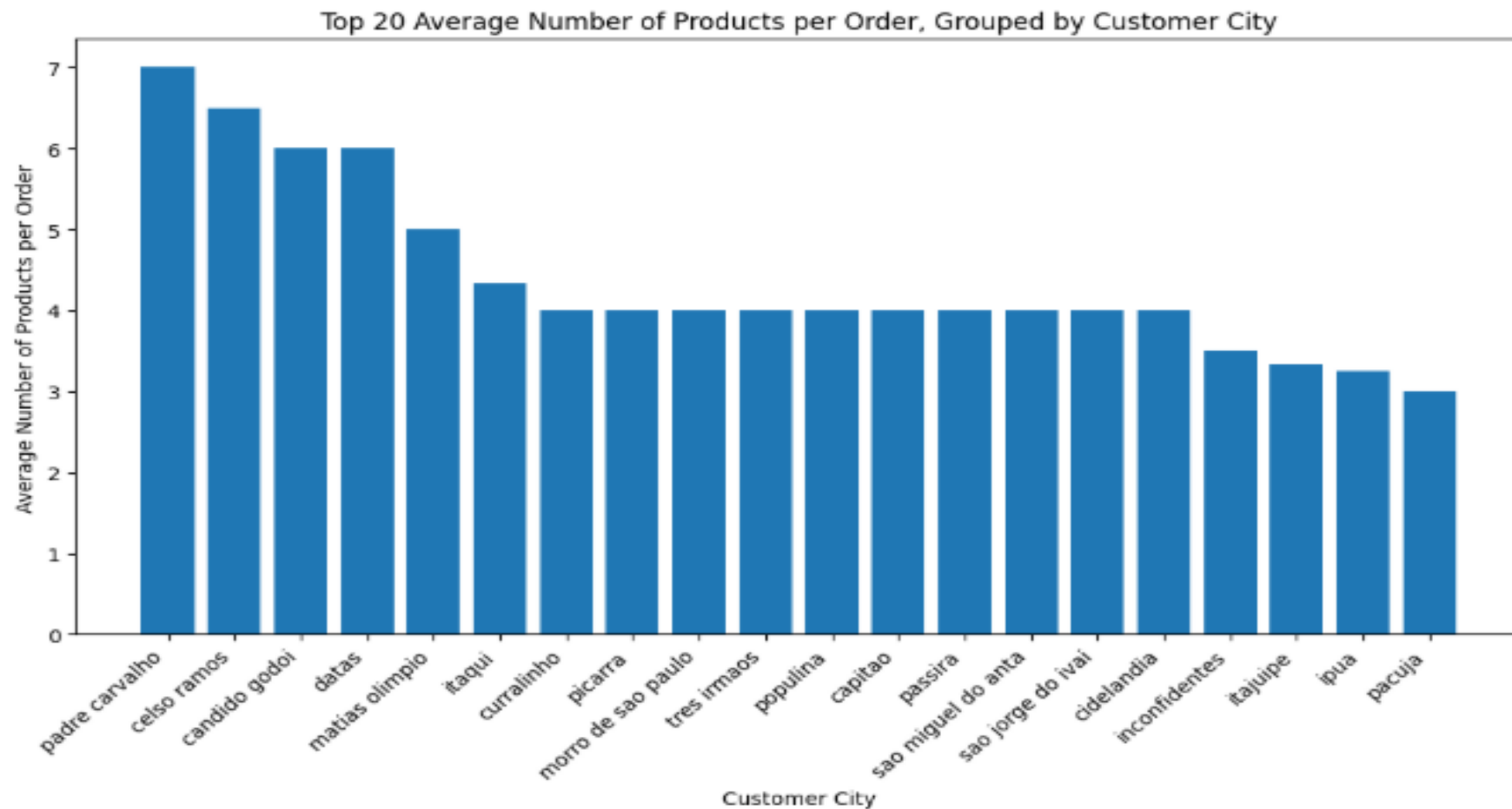
SQL Code

```
with count_per_order as
(select o.order_id, o.customer_id, count(oi.order_id) as order_count
 from orders as o
 join order_items as oi
 on o.order_id= oi.order_id
 group by o.order_id, o.customer_id)

select c.customer_city, round(avg(cp.order_count), 2) as avg_order
 from customers as c
 join count_per_order as cp
 on c.customer_id= cp.customer_id
 group by c.customer_city
 order by avg_order desc;
```

customer_city	avg_order
Padre Carvalho	7.00
Celso Ramos	6.50
Candido Godoi	6.00
Datas	6.00
Matias Olimpio	5.00
Morro De Sao Paulo	4.00
Currallinho	4.00
Teixeira Soares	4.00
Cidelandia	4.00
Picarra	4.00

Chart for average number of products per order, grouped by customer city.



C. Calculate the percentage of total revenue contributed by each product category

Python Code

```
rev_product_category= df_merged.groupby('product_category')['price'].sum().reset_index(name= 'revenue')
rev_product_category['product_category']= rev_product_category['product_category'].astype(str).str.lower().str.replace(' ', '_').
rev_product_category

total_revenue= df_merged['price'].sum()

rev_product_category['percentage_revenue']= round((rev_product_category['revenue']/total_revenue)*100, 2)
rev_product_category= rev_product_category.sort_values(by='percentage_revenue', ascending=False).reset_index(drop=True)

rev_product_category
```

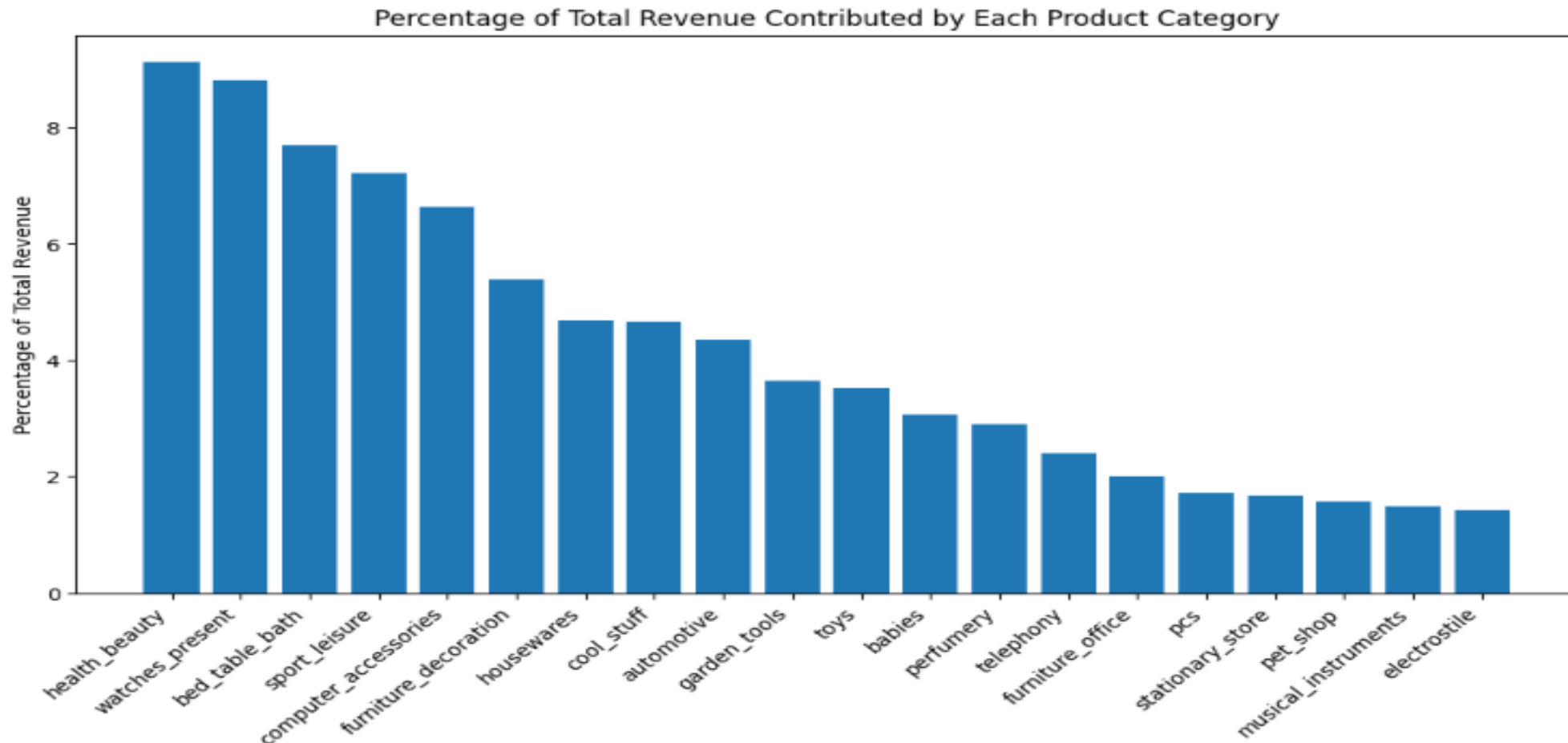
	product_category	revenue	revenue_per_category
30	HEALTH BEAUTY	1258681.34	9.26
45	Watches present	1205005.68	8.87
49	bed table bath	1036988.68	7.63
68	sport leisure	988048.97	7.27
53	computer accessories	911954.32	6.71

SQL Code

```
select p.product_category, round(sum(oi.price),2) as revenue,
round((sum(oi.price)/ (select sum(price) from order_items))*100, 2) as revenue_per_category
from products as p
join order_items as oi
on p.product_id= oi.product_id
group by p.product_category
order by revenue_per_category desc;
```

product_category	revenue	revenue_per_category
Health Beauty	1258681.34	9.26
Watches Present	1205005.68	8.87
Bed Table Bath	1036988.68	7.63
Sport Leisure	988048.97	7.27
Computer Accessories	911954.32	6.71
Furniture Decoration	729762.49	5.37
Cool Stuff	635290.85	4.67
Housewares	632248.66	4.65
Automotive	592720.11	4.36
Garden Tools	485256.46	3.57

Chart for percentage of total revenue contributed by each product category

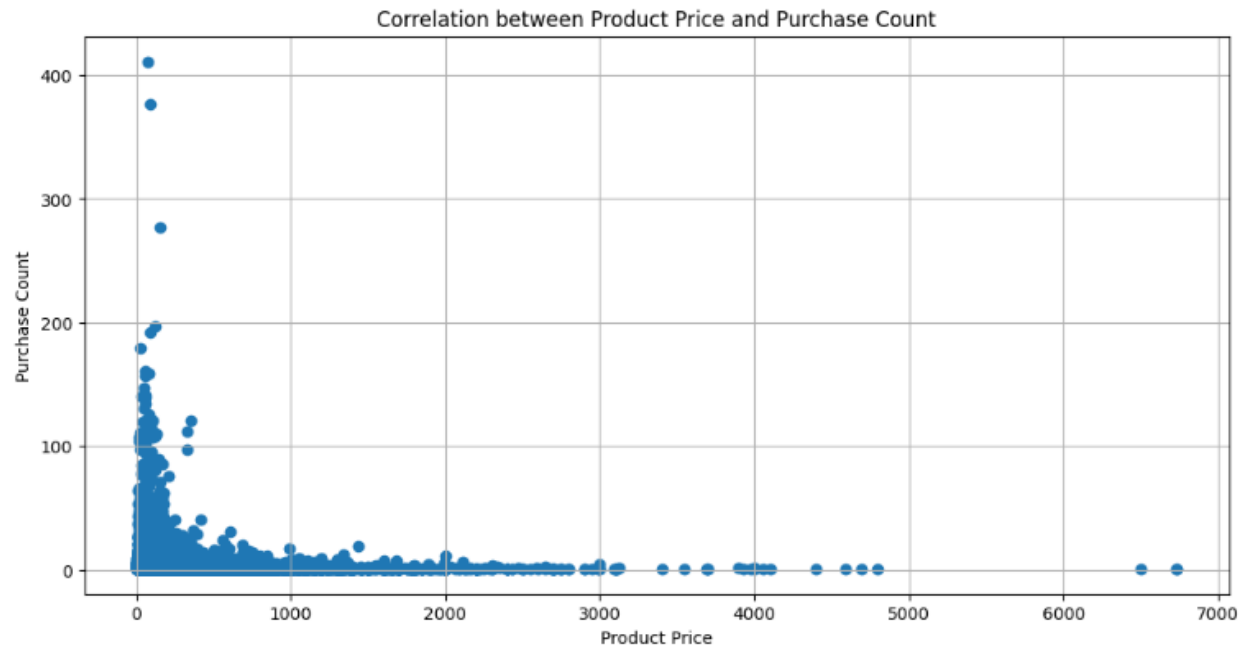


D. Identify the correlation between product price and the number of times a product has been purchased

Python Code

```
product_sales = (  
    df_merged  
    .groupby(['product_id', 'price'])  
    .agg(purchase_count=('order_id', 'count'))  
    .reset_index()  
  
    correlation = product_sales['price'].corr(product_sales['purchase_count'])  
    print(f"Correlation between product price and purchase count: {correlation}")
```

Correlation between product price and purchase count: -0.044187682605407216



SQL Code

```
select p.product_category, count(oi.product_id) as count_of_products,  
       round(avg(oi.price), 2) as avg_price  
from products as p  
join order_items as oi  
on p.product_id= oi.product_id  
group by p.product_category;
```

product_category	count_of_products	avg_price
Health Beauty	9670	130.16
Sport Leisure	8641	114.34
Cool Stuff	3796	167.36
Computer Accessories	7827	116.51
Watches Present	5991	201.14
Housewares	6964	90.79
Electronics	2767	57.91
Toys	4117	117.55
Bed Table Bath	11115	93.3
Games Consoles	1137	138.49

E. Calculate the total revenue generated by each seller and rank them by revenue.

Python Code

```
seller_revenue = df_sellers_ranks.groupby('seller_id')['price'].sum().reset_index(name='total_revenue')
```

```
seller_revenue['rank'] = seller_revenue['total_revenue'].rank(ascending=False).astype(int)
```

```
seller_revenue = seller_revenue.sort_values(by='total_revenue', ascending=False)
```

```
seller_revenue
```

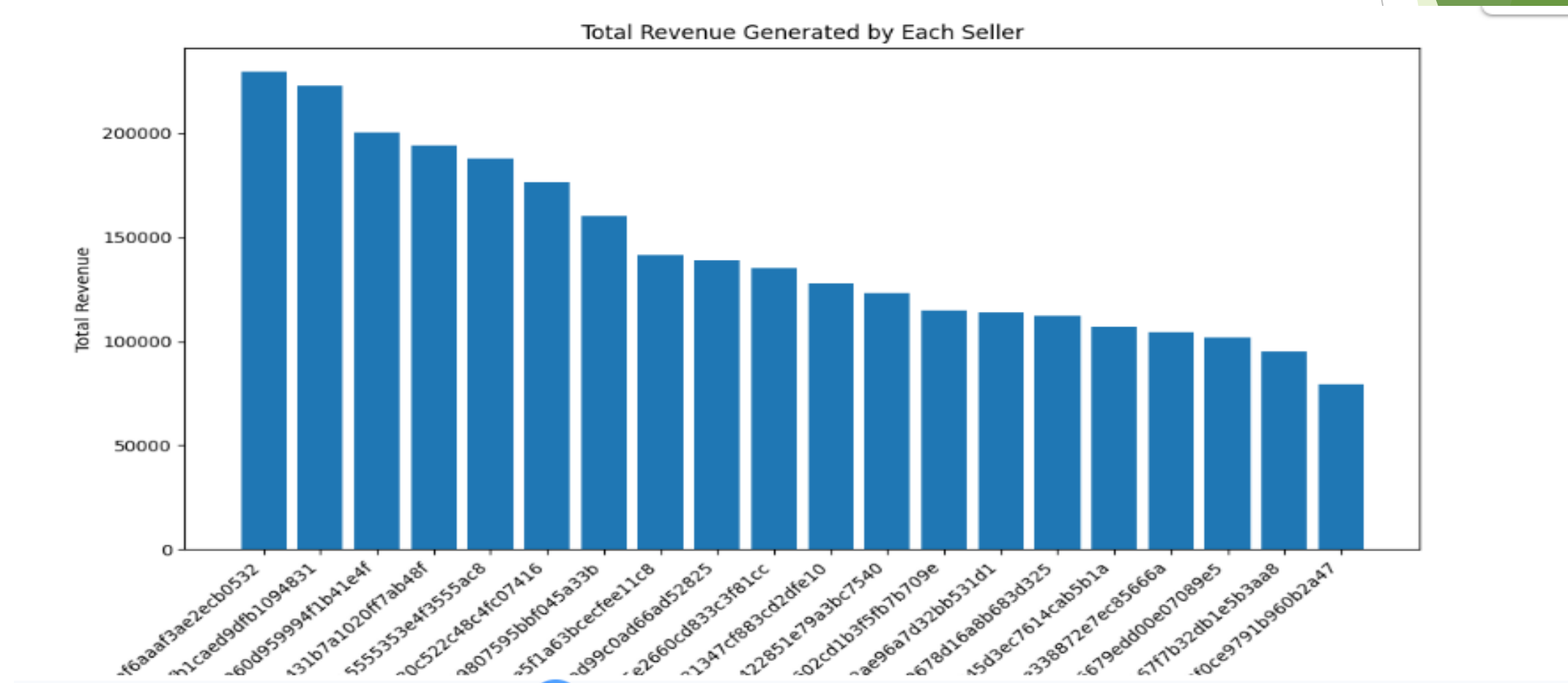
	seller_id	total_revenue	rank
857	4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1
1013	53243585a1d6dc2643021fd1853d8905	222776.05	2
881	4a3ca9315b744ce9f8e9374361493884	200472.92	3
3024	fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
1535	7c67e1448b00f6e969d365cea6b010ab	187923.89	5
...	...	...	...
627	34aefe746cd81b7f3b23253ea28bef39	8.00	3091
1370	702835e4b785b67a084280efca355756	7.60	3092
373	1fa2d3def6adfa70e58c276bb64fe5bb	6.90	3093
1465	77128dec4bec4878c37ab7d6169d6f26	6.50	3094
2519	cf6f6bc4df3999b9c6440f124fb2f687	3.50	3095

SQL Code

```
select seller_id, total_sales, rank() over (order by total_sales desc) as ranks
from (
select s.seller_id, round(sum(oi.price), 2) as total_sales
from sellers as s
join order_items as oi
on s.seller_id = oi.seller_id
group by s.seller_id) t
order by ranks;
```

seller_id	total_sales	ranks
4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1
53243585a1d6dc2643021fd1853d8905	222776.05	2
4a3ca9315b744ce9f8e9374361493884	200472.92	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
7c67e1448b00f6e969d365cea6b010ab	187923.89	5
7e93a43ef30c4f03f38b393420bc753a	176431.87	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.57	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53	8
1025f0e2d44d7041d6cf58b6550e0bfa	138968.55	9
955fee9216a65b617aa5c0531780ce60	135171.7	10

Chart for the total revenue generated by each seller and rank them by revenue





ADVANCED PROBLEMS



A. Calculate the moving average of order values for each customer over their order history

Python Code

```
df = df_payments.merge(df_orders, on="order_id")

df = (
    df[["customer_id", "order_purchase_timestamp", "payment_value"]]
    .rename(columns={"payment_value": "payment"})
    .sort_values(["customer_id", "order_purchase_timestamp"])
)

df["mov_avg"] = (
    df.groupby("customer_id")["payment"]
    .rolling(window=3, min_periods=1)
    .mean()
    .reset_index(level=0, drop=True)
)
```

	customer_id	order_purchase_timestamp	payment	mov_avg
84862	00012a2ce6f8dcda20d059ce98491703	2017-11-14 16:08:26	114.74	114.74
70509	000161a058600d5901f007fab4c27140	2017-07-16 09:40:32	67.41	67.41
83602	0001fd6190edaaf884bcdf3d49edf079	2017-02-28 11:06:43	195.42	195.42
90638	0002414f95344307404f0ace7a26f1d5	2017-08-16 13:09:20	179.35	179.35
39592	000379cdec625522490c315e70c7a9fb	2018-04-02 13:42:17	107.01	107.01

SQL Code

```
SELECT customer_id, order_purchase_timestamp, payment,
       AVG(payment) OVER (PARTITION BY customer_id ORDER BY order_purchase_timestamp
       ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS mov_avg
FROM
(
    SELECT orders.customer_id, orders.order_purchase_timestamp,
           payments.payment_value AS payment
    FROM payments
    JOIN orders ON payments.order_id = orders.order_id
) AS a;
```

customer_id	order_purchase_timestamp	payment	mov_avg
00012a2ce6f8dcda20d059ce98491703	2017-11-14	114.74	114.74
000161a058600d5901f007fab4c27140	2017-07-16	67.41	67.41
0001fd6190edaaf884bcdf3d49edf079	2017-02-28	195.42	195.42
0002414f95344307404f0ace7a26f1d5	2017-08-16	179.35	179.35
000379cdec625522490c315e70c7a9fb	2018-04-02	107.01	107.01
0004164d20a9e969af783496f3408652	2017-04-12	71.8	71.8
000419c5494106c306a97b5635748086	2018-03-02	49.4	49.4
00046a560d407e99b969756e0b10f282	2017-12-18	166.59	166.59
00050bf6e01e69d5c0fd612f1bcfb69c	2017-09-17	85.23	85.23
000500--f0-f411740765--22275120	2018-08-11	1255.71	1255.71

B. Calculate the cumulative sales per month for each year

Python Code

```
df_cumm_sales = df_orders.merge(df_order_items, on="order_id")

df_cumm_sales["sales_year"] = df_cumm_sales["order_purchase_timestamp"].dt.year
df_cumm_sales["sales_month"] = df_cumm_sales["order_purchase_timestamp"].dt.month

monthly_sales = (
    df_cumm_sales.groupby(["sales_year", "sales_month"])["price"].sum().round(2).reset_index(name="monthly_sales")
)

monthly_sales = monthly_sales.sort_values(["sales_year", "sales_month"])

monthly_sales["cumulative_sales"] = (
    monthly_sales.groupby("sales_year")["monthly_sales"].cumsum().round(2)
)
```

	sales_year	sales_month	monthly_sales	cumulative_sales
0	2016	9	267.36	267.36
1	2016	10	49507.66	49775.02
2	2016	12	10.90	49785.92
3	2017	1	120312.87	120312.87
4	2017	2	247303.02	367615.89
5	2017	3	374344.30	741960.19
6	2017	4	359927.23	1101887.42
7	2017	5	506071.14	1607958.56
8	2017	6	433038.60	2040997.16
9	2017	7	498031.48	2539028.64
10	2017	8	573971.68	3113000.32

SQL Code

```
with monthly_sales as
(
    select extract(year from o.order_purchase_timestamp) as sales_year,
           extract(month from o.order_purchase_timestamp) as sales_month,
           round(sum(oi.price), 2) as Monthly_Sales
    from orders as o
    join order_items as oi
    on o.order_id = oi.order_id
    group by extract(year from o.order_purchase_timestamp), extract(month from o.order_purchase_timestamp)
    order by extract(year from o.order_purchase_timestamp), extract(month from o.order_purchase_timestamp)

    select sales_year, sales_month, Monthly_Sales,
           ROUND(SUM(monthly_sales) OVER (PARTITION BY sales_year ORDER BY sales_month
                                           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 2) AS cumulative_sales
    from monthly_sales
    order by sales_year, sales_month;
```

sales_year	sales_month	Monthly_Sales	cumulative_sales
2016	9	267.36	267.36
2016	10	49507.66	49775.02
2016	12	10.9	49785.92
2017	1	120312.87	120312.87
2017	2	247303.02	367615.89
2017	3	374344.3	741960.19
2017	4	359927.23	1101887.42
2017	5	506071.14	1607958.56
2017	6	433038.6	2040997.16
2017	7	498031.48	2539028.64
2017	8	573971.68	3113000.32

C. Calculate the year-over-year growth rate of total sales

Python Code

```
df_yoy = df_orders.merge(df_order_items, on='order_id')
df_yoy['order_purchase_timestamp'] = pd.to_datetime(df_yoy['order_purchase_timestamp'])
df_yoy['sales_year'] = df_yoy['order_purchase_timestamp'].dt.year

year_sales = (df_yoy.groupby('sales_year')['price']
               .sum()
               .round(2)
               .reset_index(name='total_sales')
               .sort_values('sales_year'))

year_sales['previous_year_sales'] = year_sales['total_sales'].shift(1)

year_sales['YoY_Sale_percentage'] = (
    ((year_sales['total_sales'] - year_sales['previous_year_sales']) /
     year_sales['previous_year_sales']) * 100).round(2)

year_sales.rename(columns={'sales_year': 'curr_year_sales'}, inplace=True)

year_sales
```

	curr_year_sales	total_sales	previous_year_sales	YoY_Sale_percentage
0	2016	49785.92	NaN	NaN
1	2017	6155806.98	49785.92	12264.55
2	2018	7386050.80	6155806.98	19.99

SQL Code

```
with year_sales as
(
    select extract(year from o.order_purchase_timestamp) as sales_year,
           round(sum(oi.price), 2) as total_sales
    from orders as o
         join order_items as oi
         on o.order_id = oi.order_id
         group by extract(year from o.order_purchase_timestamp)
         order by extract(year from o.order_purchase_timestamp))
select sales_year as curr_year_sales, total_sales,
       LAG(total_sales) OVER (ORDER BY sales_year) AS previous_year_sales,
       round(
           (total_sales - LAG(total_sales) OVER (ORDER BY sales_year))
           / LAG(total_sales) OVER (ORDER BY sales_year) * 100,
           2) as YoY_Sale_percentage
FROM year_sales
ORDER BY
    sales_year;
```

curr_year_sales	total_sales	previous_year_sales	YoY_Sale_percentage
2016	49785.92	NULL	NULL
2017	6155806.98	49785.92	12264.55
2018	7386050.8	6155806.98	19.99

D. Identify the top 3 customers who spent the most money in each year

Python Code

```
df_top_3_cust = df_orders.merge(df_order_items, on="order_id")
df_top_3_cust["sales_year"] = df_top_3_cust["order_purchase_timestamp"].dt.year

customer_yearly_spend = (df_top_3_cust.groupby(["sales_year", "customer_id"])["price"].sum()
    .round(2)
    .reset_index(name="total_spent"))

customer_yearly_spend["spend_rank"] = (customer_yearly_spend
    .groupby("sales_year")["total_spent"]
    .rank(method="dense", ascending=False))

top_customers = (customer_yearly_spend[customer_yearly_spend["spend_rank"] <= 3]
    .sort_values(["sales_year", "spend_rank"]))

top_customers
```

sales_year	customer_id	total_spent	spend_rank
2016	a9dc96b027d1252bbac0a9b72d837fc6	1399.00	1.0
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99	2.0
2016	4a06381959b6670756de02e07b83815f	1199.00	3.0
2017	1617b1357756262bfa56ab541c47bc16	13440.00	1.0
2017	c6e2731c5b391845f6800c97401a43a9	6735.00	2.0
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499.00	3.0
2018	ec5b2ba62e574342386871631fafd3fc	7160.00	1.0
2018	f48d464a0baaea338cb25f816991ab1f	6729.00	2.0
2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.90	3.0

SQL Code

```
WITH customer_yearly_spend AS ( SELECT YEAR(o.order_purchase_timestamp) AS sales_year,
    o.customer_id, ROUND(SUM(oi.price), 2) AS total_spent
    FROM orders o
    JOIN order_items oi
    ON o.order_id = oi.order_id
    GROUP BY YEAR(o.order_purchase_timestamp), o.customer_id),
ranked_customers AS ( SELECT sales_year, customer_id, total_spent,
    DENSE_RANK() OVER (PARTITION BY sales_year ORDER BY total_spent DESC) AS spend_rank
    FROM customer_yearly_spend)
SELECT sales_year, customer_id, total_spent, spend_rank
FROM ranked_customers
WHERE spend_rank <= 3
ORDER BY sales_year, spend_rank;
```

sales_year	customer_id	total_spent	spend_rank
2016	a9dc96b027d1252bbac0a9b72d837fc6	1399	1
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99	2
2016	4a06381959b6670756de02e07b83815f	1199	3
2017	1617b1357756262bfa56ab541c47bc16	13440	1
2017	c6e2731c5b391845f6800c97401a43a9	6735	2
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499	3
2018	ec5b2ba62e574342386871631fafd3fc	7160	1
2018	f48d464a0baaea338cb25f816991ab1f	6729	2
2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.9	3

An aerial photograph of a multi-lane highway with several vehicles, overlaid with large, semi-transparent green geometric shapes that create a modern, abstract design. The shapes are primarily triangles and polygons in various shades of green, some overlapping each other. The highway is visible in the background, showing lane markings and a few cars.

Thank You